UNIVERSIDAD CENFOTEC

Escuela de Ingeniería del Software

TAREA 2

Principios de Programación 1

Problema seleccionado: Cajero automático

Estudiante: _____

Docente: Verónica Mora Lezcano

Sección: SCV3 / SCV6

Periodo: 2026-C3

Fecha de entrega: 25 de octubre de 2026

Documento elaborado a partir del ejercicio de cajero automático de la Tarea 1, adaptado a los requisitos de la Tarea 2.

1. Descripción del problema

El programa simula un cajero automático que permite realizar varios retiros usando un saldo inicial de ■250 000. El usuario indica si desea retirar dinero y escribe el monto. El sistema valida que el monto esté entre ■5 000 y ■100 000 y que no supere el saldo disponible. Si el retiro es válido, actualiza el saldo y aumenta el contador de retiros exitosos. El ciclo termina cuando la persona decide salir o cuando el saldo llega a cero.

Objetivo

Aplicar variables, constantes, operaciones aritméticas, condicionales y un ciclo while para resolver el problema del cajero automático y mostrar resultados claros.

2. Datos de entrada, procesamiento y salida

Tipo	Dato	Descripción
Entrada	continuar (s/n)	Indica si se inicia o se continúa con los retiros.
Entrada	monto (entero)	Cantidad de colones que se desea retirar.
Procesamiento	Validación del monto	Compara el monto con el mínimo, el máximo y el saldo.
Procesamiento	saldo = saldo - monto	Se ejecuta únicamente cuando el retiro es válido.
Salida	Mensajes y saldo actualizado	Indica si el retiro se realizó o por qué no se realizó.
Salida	Saldo restante y retiros exitosos	Resumen que se muestra al terminar.

Constantes utilizadas

Constante	Valor	Uso
SALDO_INICIAL	■250 000	Saldo con el que inicia el programa.
MINIMO	■5 000	Monto mínimo permitido por retiro.
MAXIMO	■100 000	Monto máximo permitido por retiro.

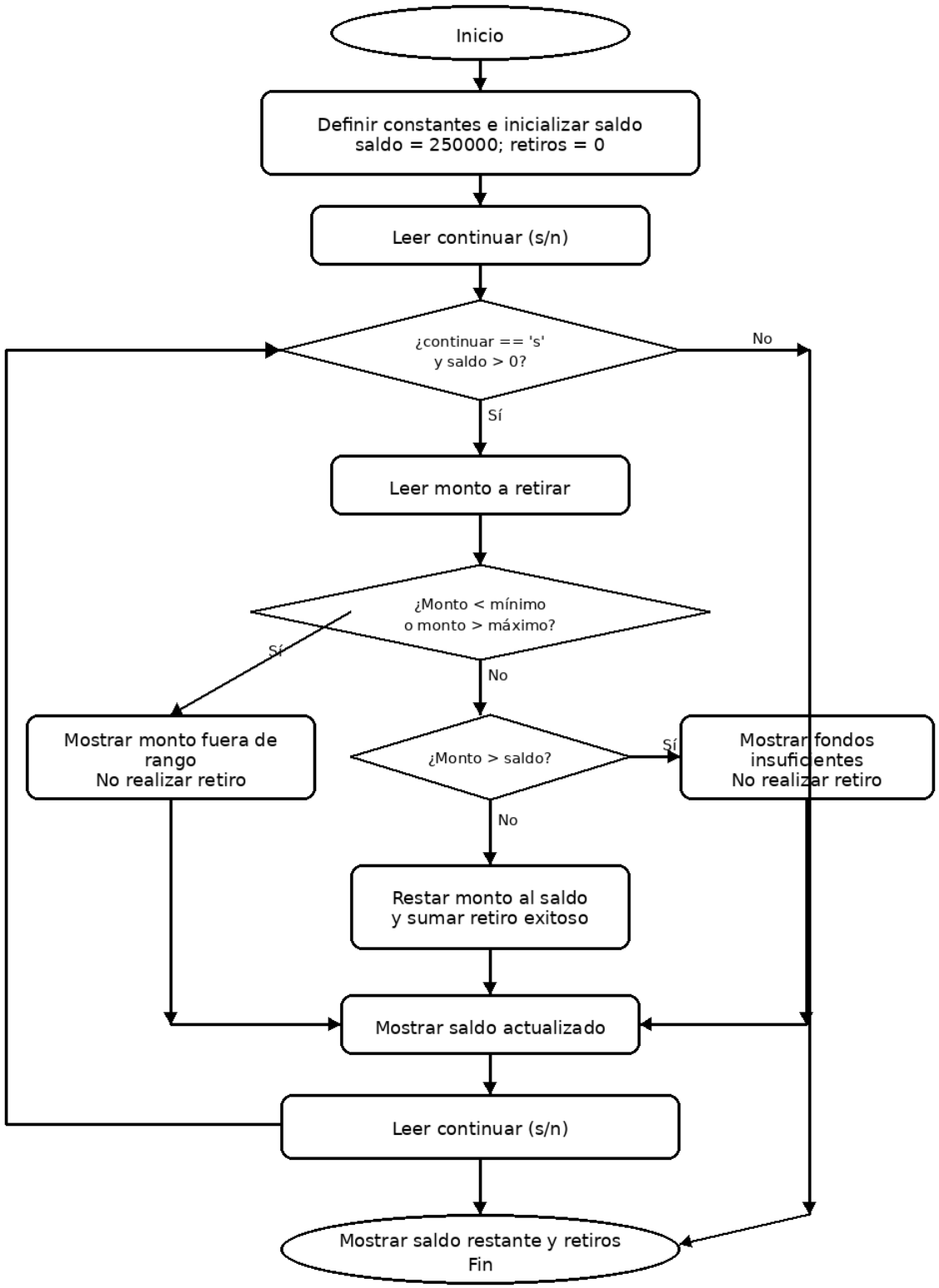
3. Algoritmo en palabras

1. Definir las constantes y asignar el saldo inicial. 2. Inicializar el contador de retiros exitosos en cero. 3. Preguntar si se desea realizar un retiro. 4. Mientras la respuesta sea “s” y el saldo sea mayor que cero, pedir el monto. 5. Si el monto está fuera del rango permitido, mostrar un mensaje sin cambiar el saldo. 6. Si el monto supera el saldo, mostrar fondos insuficientes. 7. En caso contrario, restar el monto al saldo y aumentar el contador. 8. Mostrar el saldo actualizado y preguntar si desea continuar. 9. Al salir del ciclo, mostrar el saldo restante y el total de retiros exitosos.

4. Diagrama de flujo

El diagrama representa el orden de ejecución, las decisiones para validar el retiro y el retorno al ciclo. La salida ocurre si el usuario responde “n” o si el saldo llega a cero.

DIAGRAMA DE FLUJO — CAJERO AUTOMÁTICO



5. Prueba de escritorio (traza)

Se usa el caso normal con dos retiros: ■50 000 y ■30 000. Después del segundo retiro, la persona responde “n”. La tabla muestra el estado inicial, las operaciones de cada iteración y la condición de salida.

Iteración	Variables antes / condición	¿Entra ?	Cálculos y actualizaciones	Salida / estado después
Inicial	saldo=250000; retiros=0; continuar='s'	Sí	Se lee continuar='s'. Se evalúa continuar=='s' y saldo>0: verdadero.	Entra al ciclo.
1	saldo=250000; retiros=0; monto=50000	Sí	5000 ≤ 50000 ≤ 100000 y 50000 ≤ 250000. saldo=250000-50000=200000. retiros=0+1=1.	Retiro realizado. Saldo actualizado: ■200 000. Se responde 's'.
2	saldo=200000; retiros=1; monto=30000	Sí	5000 ≤ 30000 ≤ 100000 y 30000 ≤ 200000. saldo=200000-30000=170000. retiros=1+1=2.	Retiro realizado. Saldo actualizado: ■170 000. Se responde 'n'.
Final	continuar='n'; saldo=170000; retiros=2	No	La condición continuar=='s' es falsa. El ciclo termina.	Saldo restante: ■170 000. Retiros exitosos: 2.

La variable de control continuar permite que el usuario salga del ciclo. Además, saldo > 0 impide que el programa siga solicitando retiros cuando ya no queda dinero.

6. Código fuente en Python

El archivo cajero_automatico_tarea2.py contiene el programa. Se conserva la lógica del ejercicio anterior, con constantes al inicio, nombres descriptivos, condicionales y ciclo while.

```
# Cajero automático
# Permite hacer varios retiros sobre un mismo saldo hasta que el usuario
# decida salir o hasta que ya no haya saldo disponible.

# Constantes (reglas del problema)
SALDO_INICIAL = 250000
MINIMO = 5000
MAXIMO = 100000

# Variables que cambian durante la ejecución
saldo = SALDO_INICIAL
retiros_exitosos = 0  # contador

print("=== CAJERO AUTOMÁTICO ===")
print("Saldo inicial:", saldo, "colones")

# Entrada fuera del ciclo
continuar = input("¿Desea realizar un retiro? (s/n): ").lower()

while continuar == "s" and saldo > 0:
    # Entrada dentro del ciclo
    monto = int(input("Digite el monto a retirar: "))

    if monto < MINIMO or monto > MAXIMO:
        print("Monto fuera de rango. Debe ser entre", MINIMO, "y", MAXIMO, "colones.")
        print("Retiro NO realizado.")
    elif monto > saldo:
        print("Fondos insuficientes.")
        print("Retiro NO realizado.")
    else:
        saldo = saldo - monto
        retiros_exitosos = retiros_exitosos + 1
        print("Retiro realizado.")

    print("Saldo actualizado:", saldo, "colones")

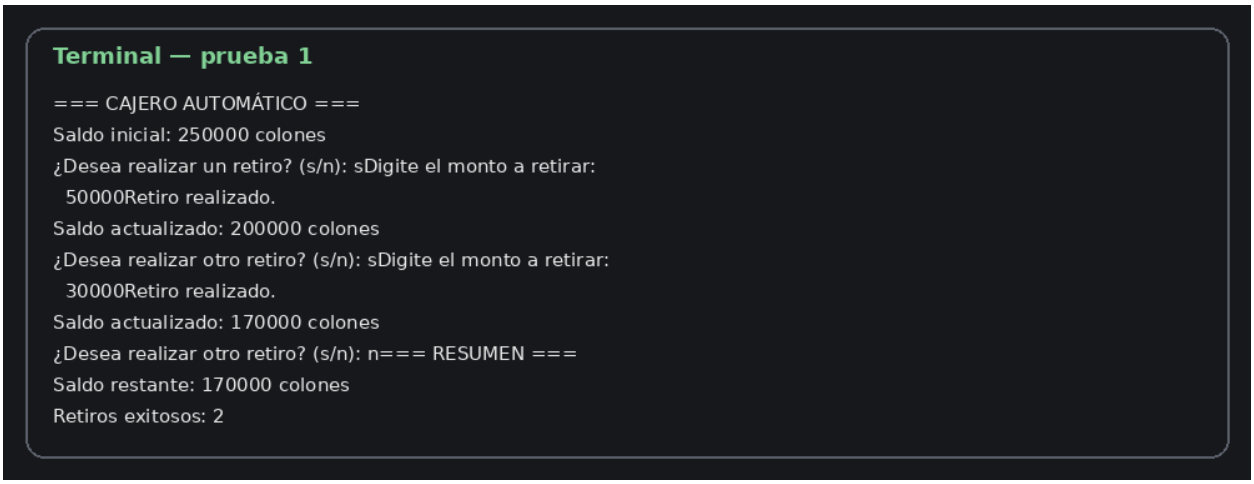
    # Entrada dentro del ciclo: cambia la variable de control
    continuar = input("¿Desea realizar otro retiro? (s/n): ").lower()

# Resultados al terminar el ciclo
print("=== RESUMEN ===")
print("Saldo restante:", saldo, "colones")
print("Retiros exitosos:", retiros_exitosos)
```

7. Pruebas del programa

Se ejecutó el archivo Python con cuatro conjuntos de entradas. Las evidencias siguientes muestran la salida de cada ejecución. Las entradas se muestran junto a las solicitudes del programa para facilitar la lectura.

Caso 1. Normal: dos retiros



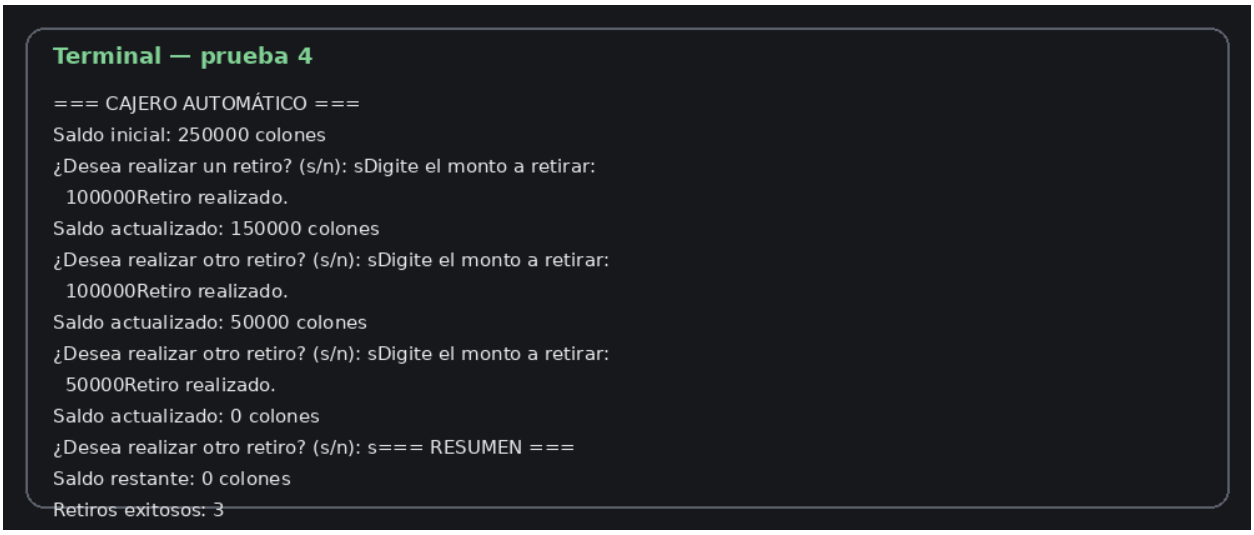
Caso 2. Cero iteraciones



Caso 3. Una iteración



Caso 4. Límite: saldo llega a cero



8. Resumen de resultados de las pruebas

Caso	Entradas principales	Resultado esperado y obtenido	Verificación
Normal	■50 000 y ■30 000; luego n	Saldo final ■170 000; 2 retiros exitosos.	Correcto
Cero iteraciones	Respuesta inicial n	Saldo final ■250 000; 0 retiros.	Correcto
Una iteración	■100 000; luego n	Saldo final ■150 000; 1 retiro exitoso.	Correcto
Límite	■100 000 + ■100 000 + ■50 000	Saldo final ■0; 3 retiros. El ciclo termina por saldo=0.	Correcto

Conclusión

El programa cumple el objetivo de simular retiros consecutivos y mantener actualizado el saldo. Las pruebas verifican que el ciclo se repita, que pueda terminar antes de iniciar, que pueda ejecutarse una sola vez y que se detenga cuando el saldo llega a cero. Las validaciones evitan aceptar montos menores al mínimo, mayores al máximo o superiores al saldo disponible.

9. Repositorio y rama

Enlace del repositorio: [Pegar aquí la URL del repositorio GitHub una vez creado].

Rama: main (si se utiliza otra rama, actualizar este dato).

Nota de entrega: la consigna pide que el repositorio incluya el archivo .py y un PDF con el enunciado, la tabla de traza y las capturas de las pruebas. El enlace del repositorio debe completarse con la URL real; no se ha inventado una dirección.

Referencias

Universidad CENFOTEC. (2026). *Consigna de la Tarea 2: Principios de programación 1* [Documento del curso].